

Course Name:	Artificial Intelligence Laboratory	Semester:	VI
Date of Performance:	_17_ / _02_ / _2026_	DIV/ Batch No:	C-3
Student Name:	Shreyans Tatiya	Roll No:	16010123325

Experiment No:4

Title: Implementation of informed search algorithms

Aim: Implementation of informed search algorithm (Greedy Best First search/A*)

Objectives of the Experiment:

By the end of this experiment, students will be able to:

1. To understand and implement informed search techniques such as Greedy Best First Search and A* algorithm for solving problem-solving tasks using heuristic functions.
2. To analyze the role of heuristic functions in guiding search and compare the performance of Greedy Best First Search and A* in terms of optimality and path cost.

COs to be achieved:

CO2: Analyse and solve problems for goal-based agent architecture (searching and planning algorithms)

Books/ Journals/ Websites referred:

1. <https://ui.adsabs.harvard.edu/abs/2022arXiv220509963S/abstract>
2. <https://verylongchicken.medium.com/greedy-best-first-search-gbfs-and-a-search-41fcc54e54c6>
3. <https://www.geeksforgeeks.org/dsa/greedy-best-first-search-algorithm/>

Theory:

Problem-Solving Agent

A problem-solving agent is designed to find solutions to well-defined problems. This agent typically follows these steps:

1. **Formulate the Problem:** Define the initial state, goal state, and possible actions.
2. **Search for a Solution:** Use an appropriate search strategy to explore the problem space.
3. **Execute the Solution:** Apply the sequence of actions derived from the search.

Informed Searching Technique

Greedy Best-First Search:

- Chooses the path that appears closest to the goal using a heuristic.
- Quick but might not find the optimal path.

A* Search:

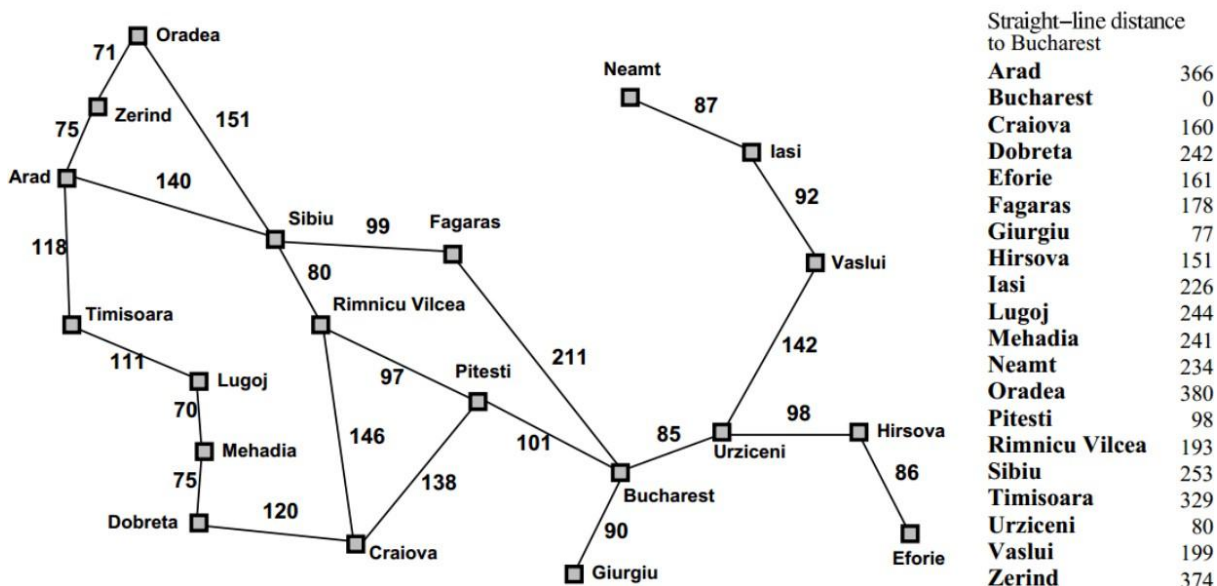
- Combines path cost and heuristic to find the best path.
- Ensures the optimal path if the heuristic is accurate.

Problem Statement:

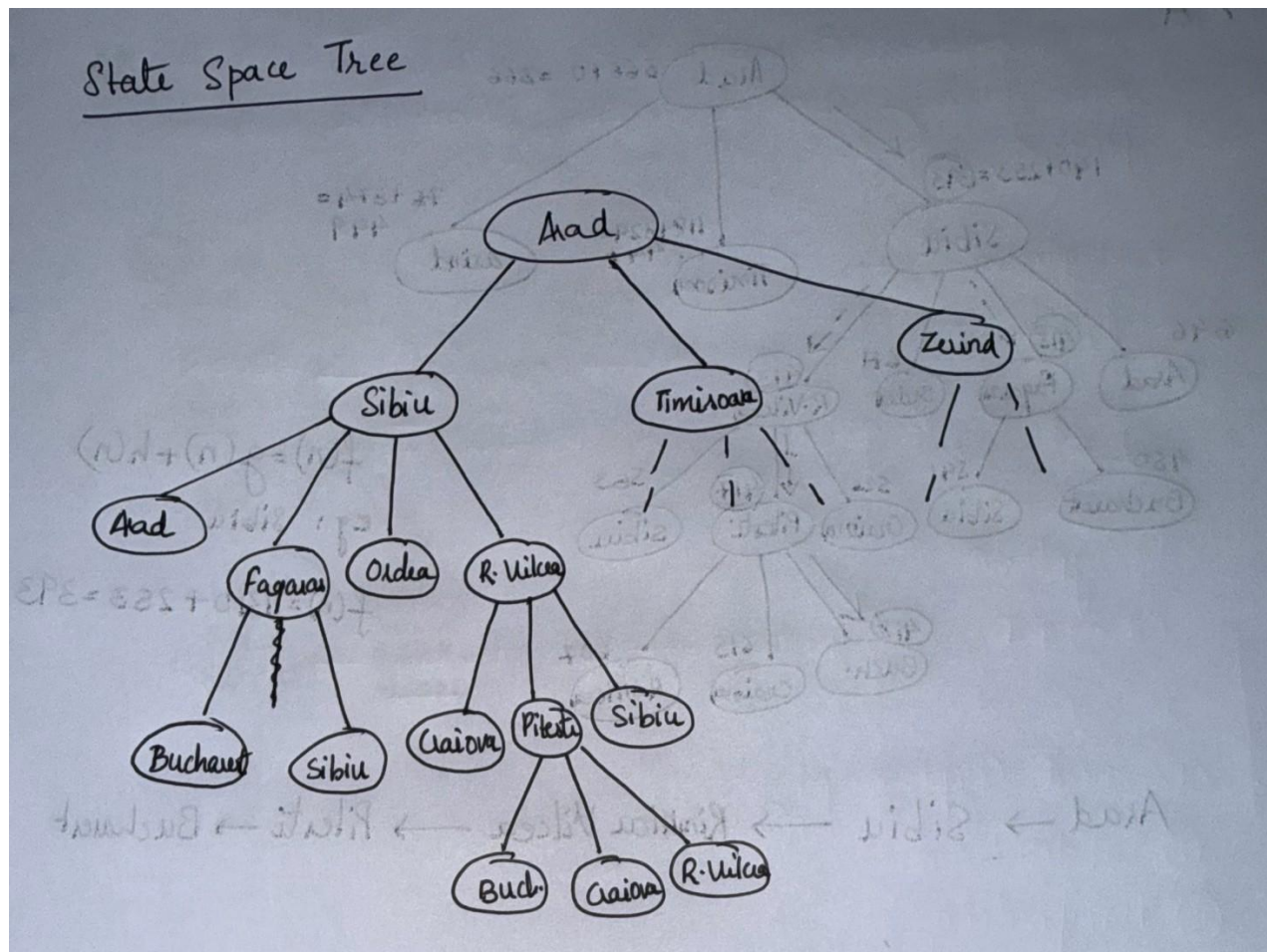
Find the shortest path from **Arad** to **Bucharest** using informed search techniques.

- Numbers on edges = actual road distance ($g(n)$)
- Numbers inside cities = straight-line distance to Bucharest (heuristic $h(n)$)

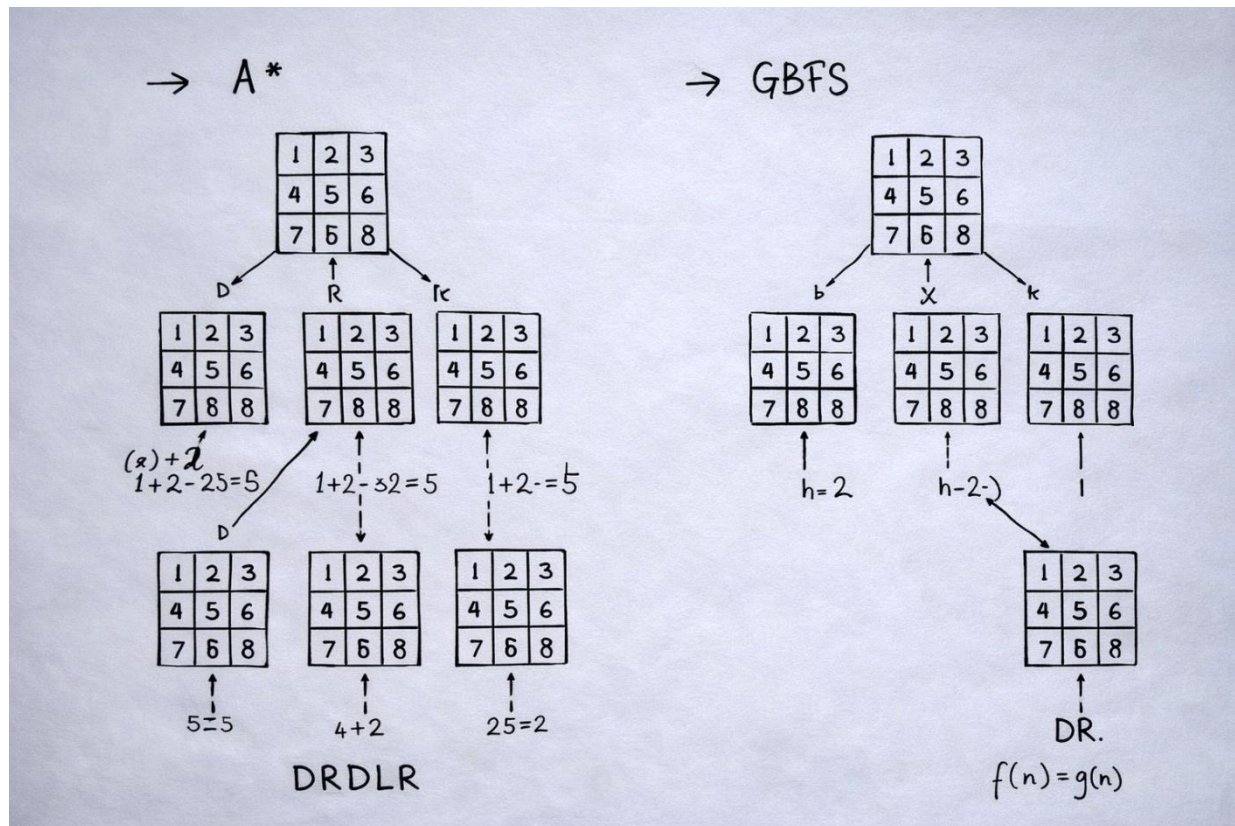
Romania with step costs in km



State-space tree:



Solution with of chosen algorithm on the state-space tree:



Comparison of performance of Greedy and A* Algorithm:

Feature	Greedy Best-First	A* Search
Uses g(n)?	No	Yes
Uses h(n)?	Yes	Yes
Optimal?	Not guaranteed	Yes (if heuristic admissible)
Complete?	Not always	Yes
Speed	Usually faster	Slightly slower
Memory Usage	Lower	Higher

Properties of A* algorithm:

- **Complete**

Will always find a solution if one exists.

- **Optimal**

Since the straight-line distance heuristic is **admissible** (never overestimates), A* guarantees shortest path.

- **Admissible Heuristic**

Straight-line distance $\leq$ actual road distance.

- **Consistent Heuristic**

For every node:

$$h(n) \leq \text{cost}(n, n') + h(n')$$

- **Time Complexity**

Exponential in worst case: $O(b^d)$

- **Space Complexity**

Stores all generated nodes $\Rightarrow$ high memory usage.

Implementation

Code:

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    vector<vector<int>> state;
    int g, h;
    string path;

    bool operator>(const Node &other) const {
        return g + h > other.g + other.h;
    }
};

vector<vector<int>> goal = {
    {1,2,3},
    {4,5,6},
    {7,8,0}
};

int dx[] = {1,-1,0,0};
int dy[] = {0,0,1,-1};
char moveChar[] = {'D','U','R','L'};

int manhattan(vector<vector<int>> &s){
    int dist = 0;
    for(int i=0;i<3;i++){
        for(int j=0;j<3;j++){
            int val = s[i][j];
            if(val==0) continue;
            int x=(val-1)/3;
            int y=(val-1)%3;
            dist += abs(x-i)+abs(y-j);
        }
    }
    return dist;
}

string serialize(vector<vector<int>> &s){
    string t="";
    for(auto &r:s)
        for(int v:r) t+=char(v+'0');
    return t;
}

void solve8Puzzle(vector<vector<int>> start){

    priority_queue<Node, vector<Node>, greater<Node>> pq;
    unordered_set<string> visited;

    pq.push({start,0,manhattan(start),""});

    while(!pq.empty()){
        Node cur = pq.top(); pq.pop();

        string key = serialize(cur.state);
        if(visited.count(key)) continue;
        visited.insert(key);

        if(cur.state == goal){
```

```
        cout<<"Solved!\nMoves: "<<cur.path<<"\n";
        cout<<"Steps = "<<cur.path.size()<<"\n";
        return;
    }

    int zx,zy;
    for(int i=0;i<3;i++)
        for(int j=0;j<3;j++)
            if(cur.state[i][j]==0) zx=i,zy=j;

    for(int k=0;k<4;k++){
        int nx=zx+dx[k], ny=zy+dy[k];
        if(nx<0||ny<0||nx>=3||ny>=3) continue;

        auto nxt = cur.state;
        swap(nxt[zx][zy], nxt[nx][ny]);

        if(!visited.count(serialize(nxt))){
            pq.push({
                nxt,
                cur.g+1,
                manhattan(nxt),
                cur.path + moveChar[k]
            });
        }
    }
}

cout<<"No solution\n";
}

int main(){

    // HARD CODED TESTCASE
    vector<vector<int>> start = {
        {1,2,3},
        {4,0,6},
        {7,5,8}
    };

    solve8Puzzle(start);
    return 0;
}
```

Output:

```
Solved!
Moves: DR
Steps = 2
```

Post Lab Subjective/Objective type Questions:

Question: Answer in brief

1. What is an informed search algorithm? How does it differ from uninformed search methods?

An **informed search algorithm** uses additional knowledge (heuristic function $h(n)$) to estimate how close a state is to the goal.

It guides the search toward promising paths instead of exploring blindly.

Examples:

- Greedy Best-First Search
- A* Search

Heuristic function:

$h(n)$ = Estimated cost from node n to goal

Feature	Informed Search	Uninformed Search
Uses heuristic?	Yes	No
Efficiency	More efficient	Less efficient
Node expansion	Directed toward goal	Explores blindly
Time complexity	Lower (usually)	Higher

2. Find a good heuristic function for following

- Monkey and Banana problem
- Travelling Salesman problem

Monkey and Banana Problem

A good heuristic is:

$h(n)$ = Number of actions remaining to get the banana

This may include:

- Moving to the box
- Pushing the box under the banana
- Climbing the box
- Grabbing the banana

This heuristic is admissible because it does not overestimate the minimum number of actions required to reach the goal.

Travelling Salesman Problem (TSP)

A good heuristic is:

$h(n)$ = Cost of the Minimum Spanning Tree (MST) of the unvisited cities

The MST gives a lower bound on the remaining travel cost.

It is admissible and helps A* efficiently estimate the minimum possible remaining distance.

3. How does A* avoid exploring unnecessarily expensive paths compared to Greedy Best First Search?

Greedy Best First Search uses:

$$f(n) = h(n)$$

It selects the node that appears closest to the goal but ignores the actual path cost from the start. This may lead to choosing expensive paths.

A* Search uses:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = actual cost from start
- $h(n)$ = estimated cost to goal

Since A* considers both the cost already travelled and the estimated remaining cost, it avoids unnecessarily expensive paths and finds the optimal solution (if the heuristic is admissible).

Conclusion:

In this experiment, we studied informed search algorithms and applied them to the Romania map problem. We compared Greedy Best First Search and A* Search to understand their working and performance differences. Greedy search selects nodes based only on the heuristic value, which may lead to suboptimal solutions. In contrast, A* Search considers both the actual path cost and the heuristic estimate, ensuring an optimal solution when the heuristic is admissible.